

Lista Spesa Facile

Guida per eseguire l'app sull'emulatore Android in Ubuntu 26.04 LTS

Questa guida porta da un Ubuntu 26.04 pulito all'app in esecuzione sull'emulatore Android, collegata al backend Laravel che gira in Docker sulla stessa macchina. Se hai già installato qualcosa (Flutter, Android SDK, Docker), salta pure i passi corrispondenti: il comando flutter doctor (cap. 4) ti aiuta a capire cosa manca.

Componente	Versione di riferimento
Sistema operativo	Ubuntu 26.04 LTS (x86_64), CPU con virtualizzazione Intel VT-x o AMD-V
Flutter	3.47 o successivo (canale stabile)
Java (JDK)	OpenJDK 17
Android SDK	piattaforma android-37, emulator, platform-tools, cmdline-tools
Backend	Docker Engine + plugin compose (MySQL, PHP-FPM, Reverb, Caddy)

Come funziona in breve

L'emulatore è una macchina virtuale con la propria rete: al suo interno l'indirizzo 10.0.2.2 corrisponde al localhost del tuo PC. L'app è già configurata per usare http://10.0.2.2 come server predefinito, quindi basta avviare il backend con Docker (porta 80 del PC) e lanciare l'app: API e WebSocket passano entrambi da Caddy sulla porta 80.

1. Pacchetti di sistema

```
sudo apt update
sudo apt install -y git curl unzip xz-utils zip libglu1-mesa \
    openjdk-17-jdk qemu-kvm cpu-checker
```

Imposta Java 17 come predefinito se ne hai più versioni installate:

```
sudo update-alternatives --config java
java -version          # deve mostrare "17.x"
```

2. Verifica dell'accelerazione hardware (KVM)

Senza KVM l'emulatore è lentissimo o non parte affatto. Controlla che sia disponibile:

```
egrep -c '(vmx|svm)' /proc/cpuinfo    # deve essere > 0
kvm-ok                                # "KVM acceleration can be used"
```

Se il numero è 0, abilita la virtualizzazione (Intel VT-x / AMD SVM) nel BIOS/UEFI. Poi aggiungi il tuo utente al gruppo **kvm**:

```
sudo usermod -aG kvm $USER
```

Importante: i gruppi nuovi (kvm, e più avanti docker) valgono solo dopo **logout e nuovo login** (o un riavvio). Verifica con il comando `groups`.

3. Installazione di Flutter

Scarica Flutter dal canale stable in `~/development`:

```
mkdir -p ~/development && cd ~/development
git clone https://github.com/flutter/flutter.git -b stable
```

Aggiungi Flutter al PATH, aprendo `~/.bashrc` e aggiungendo in fondo:

```
export PATH="$HOME/development/flutter/bin:$PATH"
```

```
source ~/.bashrc
flutter --version      # deve essere 3.47 o successivo
```

4. Installazione dell'Android SDK

Hai due strade: **Android Studio** (più semplice, con interfaccia grafica) oppure i soli **command-line tools** (più leggero). Scegline una.

Opzione A — Android Studio

```
sudo snap install android-studio --classic
android-studio
```

Al primo avvio segui la procedura guidata (*Standard*): installa SDK, emulatore e platform-tools in `~/Android/Sdk`. Poi, da *More Actions* → *SDK Manager* → *SDK Tools*, spunta anche **Android SDK Command-line Tools (latest)** e premi *Apply*.

Opzione B — Solo command-line tools

Scarica il pacchetto *Command line tools only* per Linux da developer.android.com/studio (sezione in fondo alla pagina), poi:

```
mkdir -p ~/Android/Sdk/cmdline-tools
cd ~/Android/Sdk/cmdline-tools
unzip ~/Scaricati/commandlinetools-linux-*_latest.zip
mv cmdline-tools latest
```

Variabili d'ambiente (per entrambe le opzioni)

Aggiungi in fondo a `~/.bashrc`:

```
export ANDROID_HOME="$HOME/Android/Sdk"
export PATH="$ANDROID_HOME/cmdline-tools/latest/bin:$ANDROID_HOME/platform-tools:$ANDROID_HOME/emulator
```

```
source ~/.bashrc
```

Componenti dell'SDK e licenze

```
sdkmanager "platform-tools" "emulator" "platforms;android-37.0" "build-tools;36.0.0"
flutter doctor --android-licenses      # rispondi "y" a tutte
```

I nomi esatti delle versioni cambiano nel tempo: con `sdkmanager --list | grep -E "platforms;|build-tools;"` vedi quelle disponibili. Se una versione non c'è, usa l'ultima stabile elencata; al primo `flutter run` Gradle scarica da solo eventuali componenti mancanti.

Controllo finale

```
flutter doctor
```

Le righe **Flutter** e **Android toolchain** devono avere il segno di spunta verde. Gli avvisi su Chrome, Linux desktop o Xcode si possono ignorare.

5. Creazione del dispositivo virtuale (AVD)

Scarica un'immagine di sistema x86_64 e crea l'emulatore. Elenca le immagini disponibili:

```
sdkmanager --list | grep "system-images;android-37"
```

Scegli un'immagine **google_apis_playstore** (o **google_apis**) **x86_64** e installala, sostituendo il nome con quello restituito dal comando precedente:

```
sdkmanager "system-images;android-37.2;google_apis_playstore_ps16k;x86_64"

avdmanager create avd -n Pixel_8 -d pixel_8 \
    -k "system-images;android-37.2;google_apis_playstore_ps16k;x86_64"
```

Con Android Studio puoi fare lo stesso da *More Actions* → *Virtual Device Manager* → +, scegliendo **Pixel 8** e l'immagine più recente.

Verifica che Flutter veda l'emulatore:

```
flutter emulators      # deve comparire Pixel_8
```

6. Avvio del backend con Docker

Installa Docker Engine seguendo la guida ufficiale per Ubuntu (docs.docker.com/engine/install/ubuntu), che include il plugin `docker compose`. Poi permetti al tuo utente di usarlo senza `sudo`:

```
sudo usermod -aG docker $USER      # poi logout e login
docker run --rm hello-world        # verifica
```

Prepara il file `.env` nella cartella del progetto:

```
cd ~/workspace/ListaSpesaFacile
cp .env.example .env
```

Apri `.env` e compila i valori vuoti. Puoi generarli così:

```
echo "base64:$(openssl rand -base64 32)" # → APP_KEY
openssl rand -hex 16 # → REVERB_APP_KEY
openssl rand -hex 16 # → REVERB_APP_SECRET
```

Scegli anche due password per DB_PASSWORD e DB_ROOT_PASSWORD. Lascia i valori locali già presenti:

```
SITE_ADDRESS=:80
APP_URL=http://localhost
REVERB_PUBLIC_HOST=localhost
REVERB_PUBLIC_PORT=80
REVERB_PUBLIC_SCHEME=http
```

REVERB_PUBLIC_HOST=localhost va bene anche per l'emulatore: l'app sostituisce automaticamente *localhost* con l'host dell'API (10.0.2.2).

Avvia i container (la prima volta la build richiede qualche minuto):

```
docker compose up -d --build
docker compose ps # 7 servizi "running"/"healthy"
curl http://localhost/api/config # deve rispondere con un JSON
```

Se la porta 80 è già occupata (es. da Apache o nginx installati sul PC), fermali con `sudo systemctl stop apache2 / nginx` prima di `docker compose up`.

Le migrazioni del database vengono eseguite automaticamente all'avvio del container app. Oltre ad API, WebSocket e database partono due servizi di supporto:

Servizio	A cosa serve
scheduler	Ogni minuto invia i promemoria delle liste in scadenza
worker	Invia le notifiche push a Firebase (in coda, senza rallentare l'app)

Notifiche push (facoltative)

Senza Firebase tutto funziona: con l'app aperta le notifiche arrivano in tempo reale e i promemoria vengono programmati sul telefono. Per riceverle anche **ad app chiusa**:

```
# 1. Console Firebase → nuovo progetto → Aggiungi app Android
# package: it.listaspesafacile.lista_spesa_facile
# scarica google-services.json in:
# app/android/app/google-services.json

# 2. Impostazioni progetto → Account di servizio → Genera nuova chiave privata
# secrets/firebase-service-account.json

# 3. riavvia e verifica
docker compose up -d
curl http://localhost/api/config # ... "push": true
```

Sull'emulatore le notifiche push funzionano solo con un'immagine che include i **Google Play Services** (come `google_apis_playstore` usata in questa guida).

7. Avvio dell'emulatore e dell'app

Avvia l'emulatore (in un terminale separato o in background):

```
flutter emulators --launch Pixel_8
# in alternativa: emulator -avd Pixel_8 &
```

Attendi che compaia la schermata home di Android, poi controlla che sia collegato:

```
flutter devices      # deve comparire "sdk gphone64 x86 64 (mobile) • emulator-5554"
```

Scarica le dipendenze e avvia l'app:

```
cd ~/workspace/ListaSpesaFacile/app
flutter pub get
flutter run
```

La prima compilazione Gradle può richiedere diversi minuti: scarica anche da sola la piattaforma Android 35 e CMake, richieste dai plugin delle notifiche. Durante l'esecuzione di `flutter run` puoi usare questi tasti nel terminale:

Tasto	Azione
r	Hot reload: applica le modifiche al codice mantenendo lo stato
R	Hot restart: riavvia l'app da zero
q	Chiude l'app e termina flutter run

Primo utilizzo

Nella schermata di accesso tocca **Registrati** e crea un account con nome, email e password. Il server predefinito è già `http://10.0.2.2`; se serve, puoi cambiarlo dalla voce **Server** della stessa schermata. Dopo l'accesso Android chiede il permesso per le **notifiche**: tocca **Consenti**, altrimenti promemoria e avvisi non compariranno.

Il login con **Google, Facebook o Amazon** non funziona in locale: i provider accettano solo callback HTTPS. Sull'emulatore usa la registrazione con email e password; il login social va provato sul VPS o tramite un tunnel HTTPS (es. `cloudflared`).

Provare il tempo reale con due emulatori

Per vedere condivisione, chat e notifiche crea un secondo AVD e avvia l'app su entrambi, con due account diversi:

```
avdmanager create avd -n Pixel_8_B -d pixel_8 \
    -k "system-images;android-37.2;google_apis_playstore_ps16k;x86_64"
flutter emulators --launch Pixel_8_B

flutter devices      # annota gli id, es. emulator-5554 e emulator-5556
flutter run -d emulator-5554    # primo terminale
flutter run -d emulator-5556    # secondo terminale
```

Poi prova le funzioni principali:

Prova	Cosa succede
Con il primo utente tocca Nuova lista , scrivi l'email del secondo in Condividi con e scegli se <i>Può modificare</i>	Il secondo riceve subito la notifica e la vede nella campanella (con il numero di non lette)
Aggiungi <i>mele, spinaci e pere</i> ; per un articolo scrivi anche il peso (es. 500 g)	Ogni prodotto riceve la sua icona (mela, foglia di verdura, pera) e la lista si ordina per reparto: Frutta (mele, pere), Verdura (spinaci). Con la matita puoi correggere il reparto
Spunta articoli	Le modifiche compaiono subito sull'altro emulatore; sotto il titolo vedi chi sta guardando la lista
Apri la chat (icona del fumetto nella lista) e scrivi un messaggio	Arriva in tempo reale all'altro utente, anche se ha la sola lettura. Tieni premuto per eliminarlo
Crea una lista tra 12 minuti con promemoria Personalizzato... 10 minuti e Chi avvisare = <i>Creatore e destinatari</i>	Dopo circa 2 minuti compare la notifica <i>Promemoria: ... La spesa è tra 10 minuti.</i>
Tocca una notifica	Si apre la lista (o la chat, per i messaggi)

Senza Firebase il promemoria è una notifica programmata sul telefono: Android può ritardarla di qualche minuto. Con Firebase lo invia il server (container scheduler) con al massimo un minuto di ritardo.

8. Risoluzione dei problemi

Sintomo	Soluzione
<i>/dev/kvm device: permission denied</i> o emulatore molto lento	Utente non nel gruppo kvm o virtualizzazione disattivata nel BIOS. Esegui <code>sudo usermod -aG kvm \$USER</code> , poi <code>logout/login</code> ; verifica con <code>kvm-ok</code> .
L'emulatore si apre con schermo nero o va in crash	Forza il rendering software: <code>emulator -avd Pixel_8 -gpu swiftshader_indirect</code> . Se non basta, avvio pulito: <code>emulator -avd Pixel_8 -wipe-data</code> .
<i>flutter devices</i> non mostra l'emulatore	<code>adb kill-server && adb start-server</code> , poi <code>adb devices</code> .
<i>Android license status unknown</i> in flutter doctor	<code>flutter doctor --android-licenses</code> e accetta tutto.
<i>cmdline-tools component is missing</i>	Installa Android SDK Command-line Tools dall'SDK Manager (o l'opzione B del cap. 4) e controlla che <code>ANDROID_HOME</code> punti a <code>~/Android/Sdk</code> .
Build Gradle fallisce per la versione di Java	Serve JDK 17: <code>flutter config --jdk-dir /usr/lib/jvm/java-17-openjdk-amd64</code> .
Nell'app: errore di connessione al server	Controlla che il backend risponda: <code>curl http://localhost/api/config</code> . Nella voce Server deve esserci <code>http://10.0.2.2</code> , non <code>localhost</code> .

Sintomo	Soluzione
Le modifiche non arrivano in tempo reale	Verifica il container Reverb: <code>docker compose logs -f reverb</code> ; controlla che <code>REVERB_APP_KEY</code> e <code>REVERB_APP_SECRET</code> siano compilati nel <code>.env</code> , poi <code>docker compose up -d</code> .
<i>permission denied</i> su <code>docker.sock</code>	Utente non nel gruppo docker : <code>sudo usermod -aG docker \$USER</code> , poi <code>logout/login</code> .
<i>adb: device unauthorized</i>	Le immagini con Play Store chiedono di autorizzare il debug USB: sullo schermo dell'emulatore tocca Consenti (spunta <i>Consenti sempre</i>). Poi <code>adb kill-server</code> .
Le notifiche non compaiono	Permesso negato: Impostazioni Android → App → Lista Spesa Facile → Notifiche → attiva. Le notifiche nell'app (campanella) arrivano comunque.
Il promemoria non arriva	Deve cadere nel futuro (uno già passato non viene inviato). Controlla lo scheduler: <code>docker compose logs -f scheduler</code> .
Firebase configurato ma " <i>push</i> ": <i>false</i>	Il file <code>secrets/firebase-service-account.json</code> manca o non è leggibile; dopo averlo aggiunto <code>docker compose up -d</code> . Log: <code>docker compose logs -f worker</code> .
Errori strani dopo aggiornamenti di Flutter o dipendenze	<code>flutter clean && flutter pub get</code> , poi di nuovo <code>flutter run</code> .

Riepilogo: avvio quotidiano

Una volta completata l'installazione, ogni volta bastano questi comandi:

```
cd ~/workspace/ListaSpesaFacile
docker compose up -d           # 1. backend (anche worker e scheduler)
flutter emulators --launch Pixel_8  # 2. emulatore
cd app && flutter run           # 3. app
```

Per fermare tutto: `q` nel terminale di `flutter run`, chiudi la finestra dell'emulatore e poi `docker compose down` (i dati del database restano nel volume `mysql_data`).